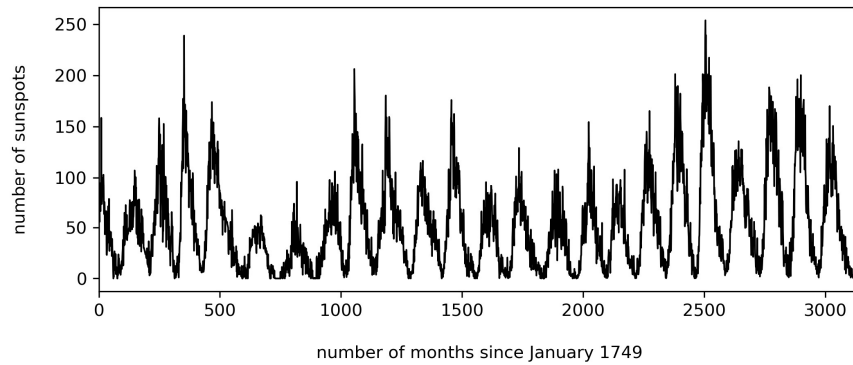


MSDM5004 Spring 2026
Homework 3 Solution

1. Detecting periodicity

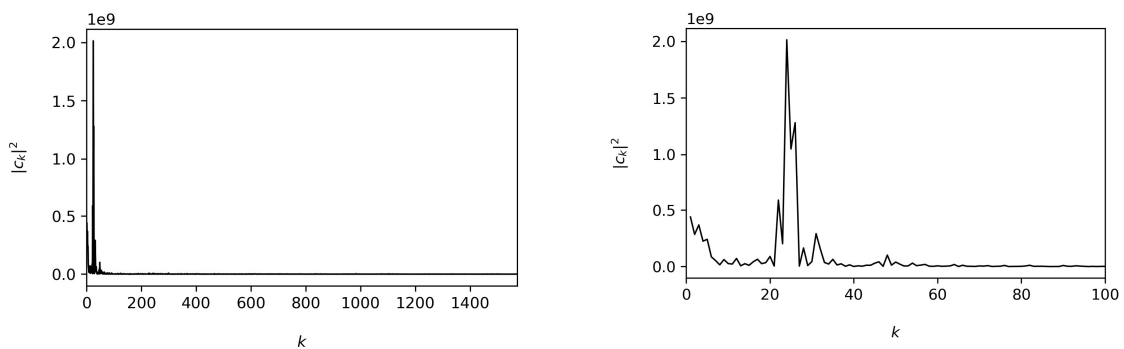
Refer to `Homework3Q1_solution.py`.

- (a) Let S be the number of sunspots.



There have been $n = 24$ troughs in $|S| = 3143$ months, so the length of one cycle is around $|S|/n = 3143/24 = 131.0$ months.

- (b) Note that c_0 is the sum of S and will dominate the power spectrum $|c_k|^2$, so the following graphs are obtained after removing c_0 . The left one shows all available indices k , whereas the right one zooms into $0 \leq k \leq 100$.

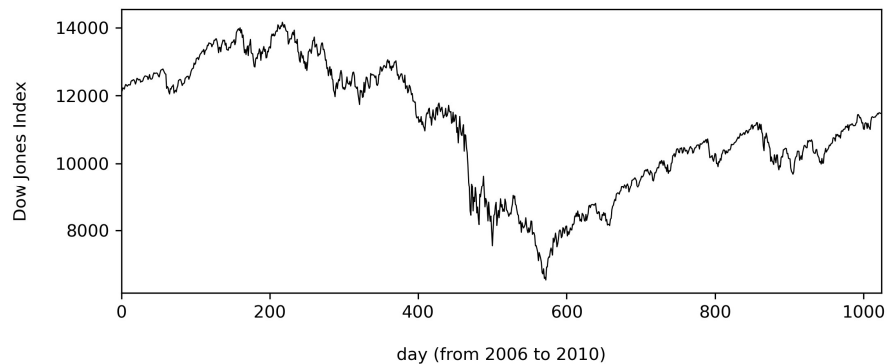


- (c) The power spectrum peaks at $k_p = 24$, and the corresponding period is $|S|/k_p = 3143/24 = 131.0$ months, which is identical to the answer in (a). (In fact, the sunspot cycle is known to have an average period around 11 years, i.e. 132 months.)

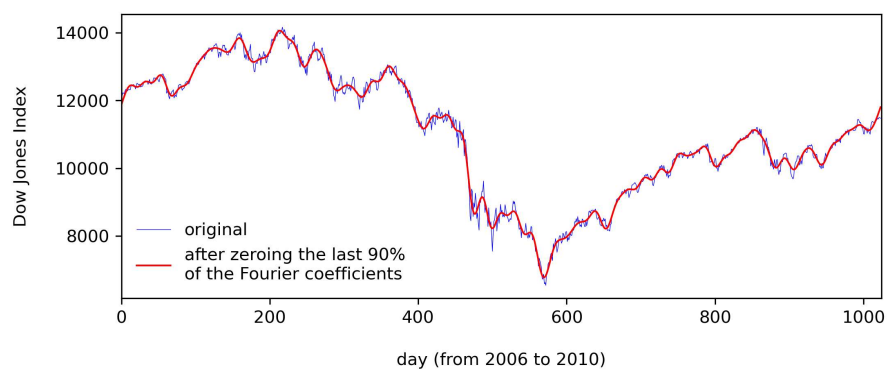
2. Fourier filtering and smoothing

Refer to Homework3Q2_solution.py.

(a)

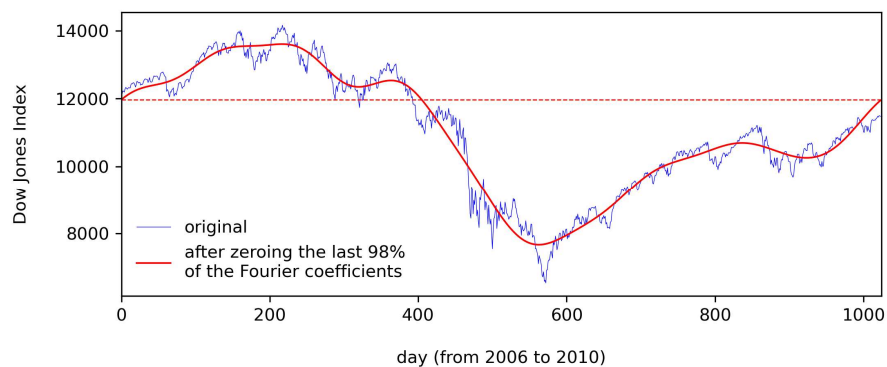


(d) The blue curve represents the original data. The red curve represents the inverse Fourier transform. The red curve looks a bit smoother than the blue curve.



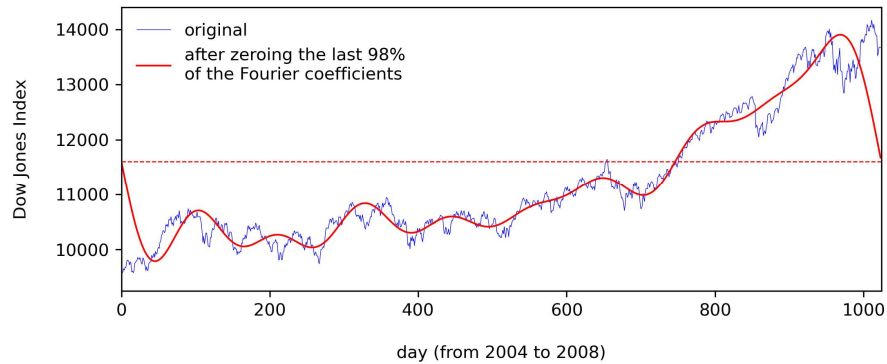
Since the last 90% of the Fourier coefficients correspond to the magnitude of the high-frequency oscillations, zeroing them removes these oscillations from the red curve and thus makes it look smoother.

(e)



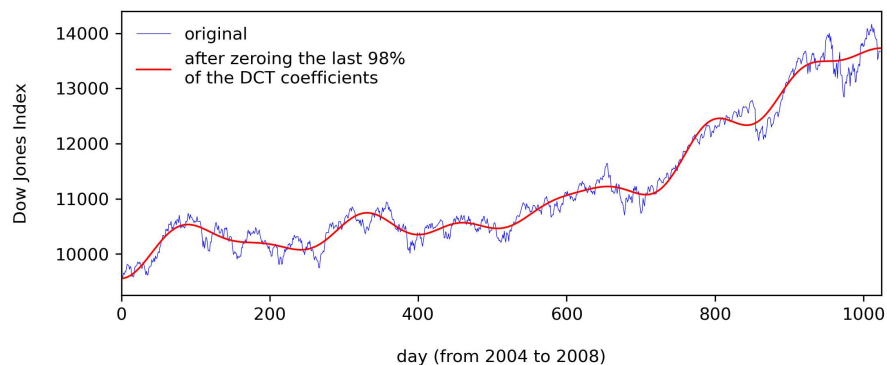
As even less Fourier coefficients are kept, the red curve looks even smoother. It has discarded most oscillations of the data and only preserves the trends that have very low frequencies. In addition, observe how the two ends of the red curve are on the same level.

(f)



Observe how the two ends of the red curve are, again, on the same level, but this is clearly an artifact since they deviate much from the two ends of the blue curve.

(g)



Observe how the DCT makes the red curve resemble the blue curve better.

3. Deblurring by circular deconvolution

Refer to `Homework3Q3_solution.py`.

(a) See below.

(b) Since $f(x, y)$ is a two-dimensional function, its periodic summation $f_M[x, y]$ is a double summation. Its period is $M = 1024$ and equal to the length (and width) of the photo.

$$f_M[x, y] = \sum_{k_x=-\infty}^{+\infty} \sum_{k_y=-\infty}^{+\infty} f[x + k_x M, y + k_y M]$$

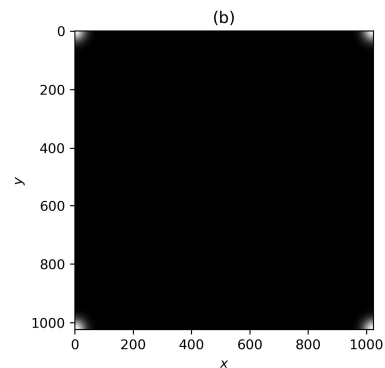
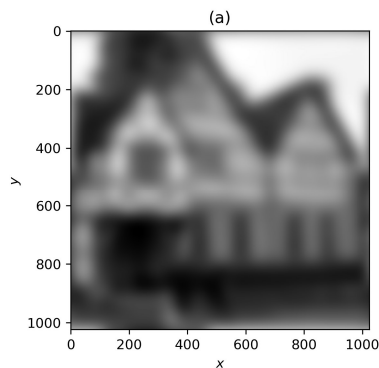
Theoretically speaking, both k_x and k_y should go from $-\infty$ and $+\infty$; but practically speaking, $f(x, y)$ is exponentially small when (x, y) is far from the origin, so only four terms are significantly larger than 0 and can contribute to the summation:

- $f[x, y]$ (when $x \rightarrow 0$ and $y \rightarrow 0$)
- $f[x - M, y]$ (when $x \rightarrow M$ and $y \rightarrow 0$),
- $f[x, y - M]$ (when $x \rightarrow 0$ and $y \rightarrow M$), and
- $f[x - M, y - M]$ (when $x \rightarrow M$ and $y \rightarrow M$).

Therefore,

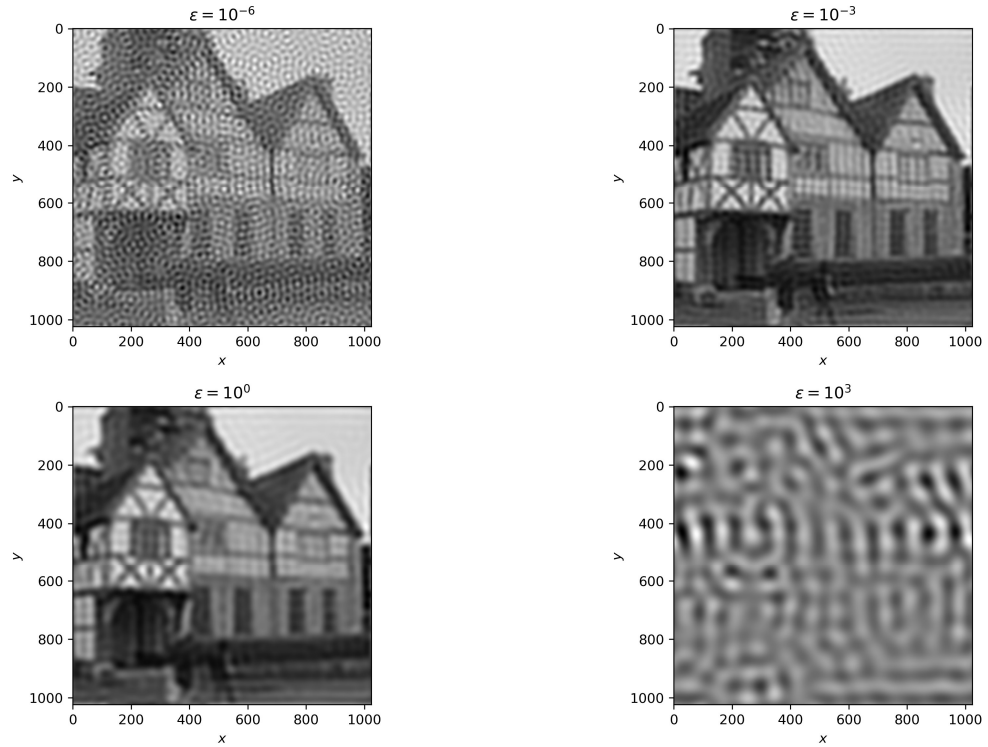
$$f_M[x, y] = \sum_{k_x=-1}^0 \sum_{k_y=-1}^0 f[x + k_x M, y + k_y M] .$$

Its graph is shown below. Notice that it remains the same after rotating 180° , so $f_M(x, y) = f_M(-x, -y)$ and f_M is an even function.



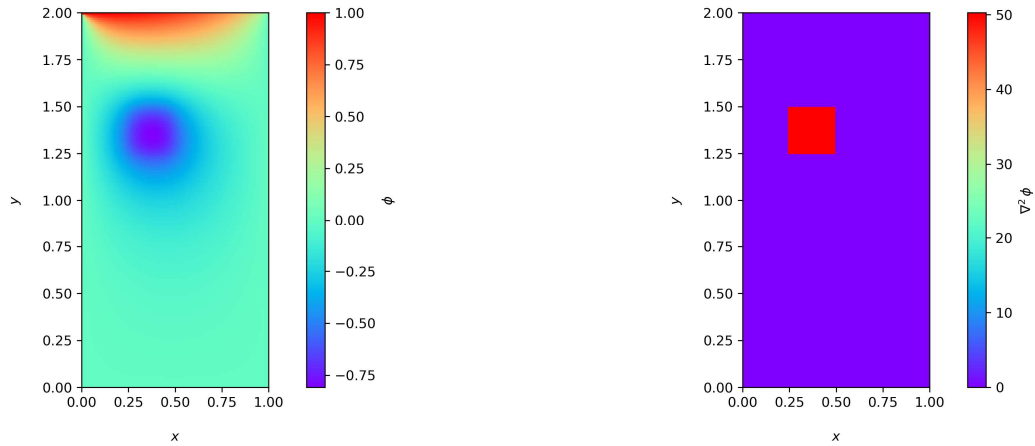
(c) First, the two-dimensional Fourier transform is used here. Second, since f_M is an even function, its Fourier transform F has real coefficients only. Third, only the coefficients larger than ε are used in $G = J/F$, where J is the Fourier transform of the blurred photo j . The following graphs use $\varepsilon = 10^{-6}, 10^{-3}, 10^0, 10^3$.

- If ε is too small, extremely large values pop up in G , so G is dominated by certain frequencies. Therefore, the deblurred photo g , which is the inverse Fourier transform of G , shows obvious periodic changes in brightness and looks pixelated.
- If ε is too large, the division J/F can be hardly carried out, so $G \approx J$, making the “deblurred” photo g as blurred as the original one.



4. The Poisson equation and the spectral method

Refer to `Homework3Q4_solution.py`. The graph on the left plots ϕ , while the graph on the right verifies by finite differencing that ϕ indeed satisfies the Poisson equation.



Note that the program computes $\rho_{\ell j}$ and $\phi_{\ell j}$ (i.e. `rlj` and `plj`) instead of $\rho_{j\ell}$ and $\phi_{j\ell}$. Similarly, their DSTs become $\tilde{\rho}_{nm}$ and $\tilde{\phi}_{nm}$ (i.e. `Rnm` and `Pnm`).

Remarks. Note that the DST formulae

$$\tilde{\rho}_{nm} = \sum_{\ell=1}^{L-1} \sum_{j=1}^{J-1} \rho_{\ell j} \sin \frac{\pi \ell n}{L} \sin \frac{\pi j m}{J}$$

and

$$\phi_{lj} = \frac{2}{L} \frac{2}{J} \sum_{n=1}^{L-1} \sum_{m=1}^{J-1} \tilde{\phi}_{nm} \sin \frac{\pi \ell n}{L} \sin \frac{\pi j m}{J}$$

keep computing two expensive terms

$$\sin \frac{\pi \ell n}{L} \quad \text{and} \quad \sin \frac{\pi j m}{J}$$

for different repetitive combinations of (ℓ, n) and (j, m) . For example, the term $\sin(\pi/L)$ at $(\ell, n) = (1, 1)$ is computed $(L - 1)$ times for $\tilde{\rho}_{nm}$ and $(L - 1)$ times for ϕ_{lj} . Therefore, the program can be significantly sped up by saving and reusing these values wisely.